Programming Foundation using Python

Case Study 1: Student Performance Analysis System

A university department wants to build a small program that analyses student marks. The system should allow the user to enter marks of several students and produce statistical results such as average marks, highest marks, lowest marks, and pass/fail counts.

The program must be developed using basic programming concepts including **lists, for loops, functions, and exception handling**.

Tasks

Task 1: Accept Number of Students

Write code to ask the user to enter the total number of students.

Requirements:

- The number must be a positive integer.
- Use **exception handling** to handle invalid inputs.

Example:

Enter number of students: 5

Task 2: Input Student Marks

Create a list (array) to store marks of students.

Requirements:

- Use a **for loop** to collect marks.
- Validate each mark before storing it.
- Marks must be between **0 and 100**.

Example:

Enter mark for student 1: 75

Enter mark for student 2: 60

Enter mark for student 3: 88

Enter mark for student 4: 45

Enter mark for student 5: 90

If the user enters an invalid value:

Error: Marks must be between 0 and 100

Task 3: Create a Function to Calculate Average Marks

Create a function named:

calculate_average(marks)

Task:

- Use a **for loop** to calculate total marks.
- Divide total marks by number of students.
- Return the average.

Expected Output:

Average Marks: 71.6

Task 4: Find the Highest Marks

Create a function:

find_highest(marks)

Tasks:

- Use a loop to compare values.
- Determine the highest mark.
- Return the result.

Expected Output:

Highest Marks: 90

unext

Task 5: Find the Lowest Marks

Create a function:

find_lowest(marks)

Tasks:

- Compare values using a loop.
- Determine the lowest marks.

Expected Output:

Lowest Marks: 45

Task 6: Count Passed and Failed Students

Create a function:

count_results(marks)

Tasks:

- Count students scoring **50 or above** as pass.

- Count students scoring **below 50** as fail.

Expected Output:

Students Passed: 4

Students Failed: 1

Task 7: Display the Final Report

Display all results together:

Student Performance Report

Average Marks: 71.6

Highest Marks: 90

Lowest Marks: 45

Students Passed: 4

Students Failed: 1

Case Study 2: Inventory Stock Monitoring System

A warehouse wants to track the quantity of products in stock. The system should store product quantities, allow updates, and detect products with low stock levels.

Task 1: Create Initial Stock List

Create a list storing quantities of products.

Example:

stock = [25, 12, 5, 30, 8]

Display stock values:

Product 1 : 25

Product 2 : 12

Product 3 : 5

Product 4 : 30

Product 5 : 8

Task 2: Display Stock Function

Create a function:

display_stock(stock)

Tasks:

- Use a **for loop** to display each product and its quantity.

Task 3: Update Stock

Create a function:

update_stock(stock, product_number, quantity)

Tasks:

- Ask the user which product to update.
- Ask the new quantity.
- Update the list.

Example:

Enter product number: 3

Enter new quantity: 15

Stock updated successfully

Task 4: Detect Low Stock

Create a function:

check_low_stock(stock)

Tasks:

- Identify products with quantity **less than 10**.

Example Output:

Low Stock Alert:

Product 3

Product 5

Task 5: Error Handling

Use exception handling for:

- Invalid product number
- Negative stock value
- Non-numeric input

Example:

Error: Invalid product number

Task 6: Create Menu System

Display menu:

1. View Stock

2. Update Stock

3. Check Low Stock

4. Exit

Use a **loop** to repeat operations.

Task 7: Terminate Program

Allow user to exit the system.

Example:

Inventory system closed.

---

unext